The Ultra Scale Playbook

① Intros and Theory and Concepts

② Parallelizing the model and Increasing throughput by scaling up GPUs

③ Clear code implementation

④ Real training efficiency benchmarks

Memory usage → Hard limitation

Training with 1 GPU

Forward pass → backward pass → Optimization step
① ② ③

Batch size → Represented in terms of Tokens
↳ (bst)

training on single machine ⇒ bst = bs * seq
↑ samples ↑ Input seq length

What all stored in memory?

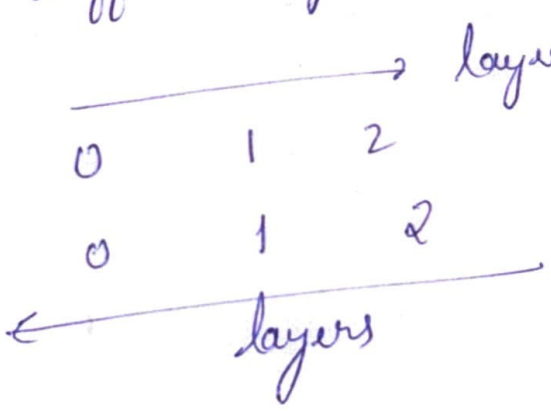
Model weights

2/3/25

Ultra Scale Playbook Lecture Video

Reliability issues :-

Each GPU see different chunk of data, and different gradient, → they gets summed up later



All reduce → distributed communication

↳ how all GPUs and use multiple function and all of them going to have same output.

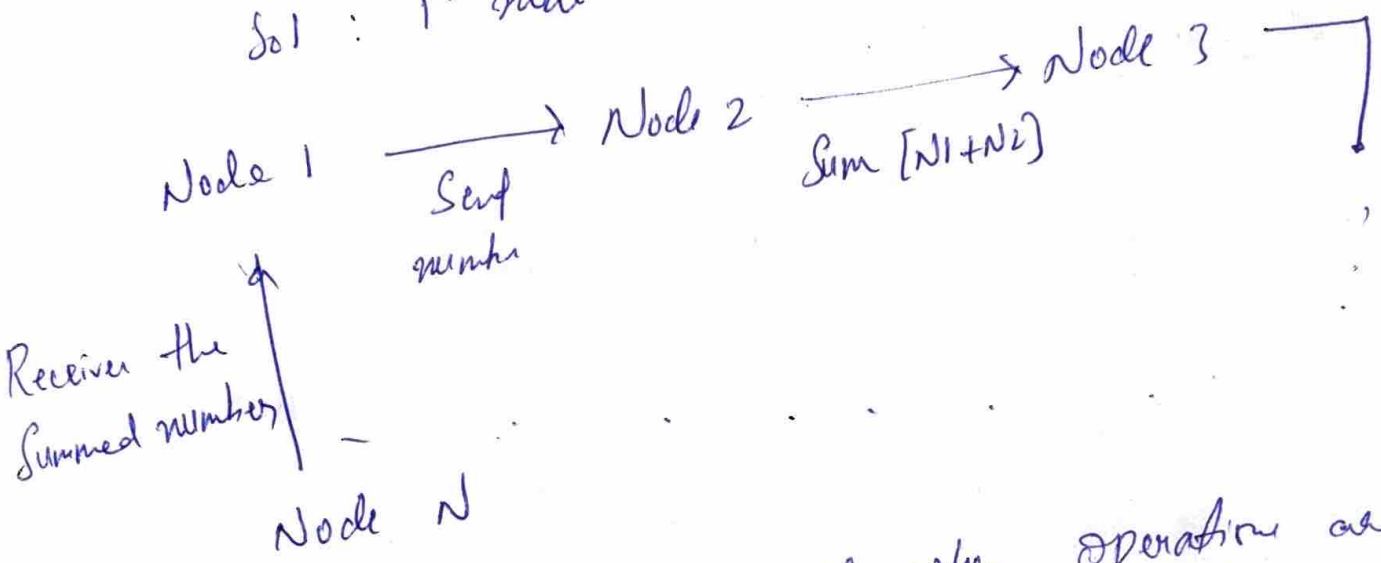
combine data present on each node using them function (Some function, that we write) which can be instant Summation or averaging

In Reduce, Result is sent to Root-Node

In All Reduce, result is sent to all node

eg: Problem = Calculate sum of numbers on each node. Nodes are connected in a ring Pattern

Sol: 1st node sends its number to neighb



Generally GPU's work well when operations are Performed on large tensors than small tensors So in the back prop we group Gradients into buckets — Called Bucketing Gradient [optimization method]

ex :- Its like Sending One Big Box instead of few small boxes (shipping)

Reduces computation Overhead and Speed up communication.

2 Important things in GPU :-

1. GPU Computation
2. GPU Communication

we always tend to improve these 2

Gradient Accumulation

each GPU see new batch of data, does gradient accumulation.

Gradient Accumulation + Data Parallelism

Careful while synchronizing Gradients

All Reduce is auto triggred after each back pass during accumulation

model.no_sync()

Global Batch Size (GBS)

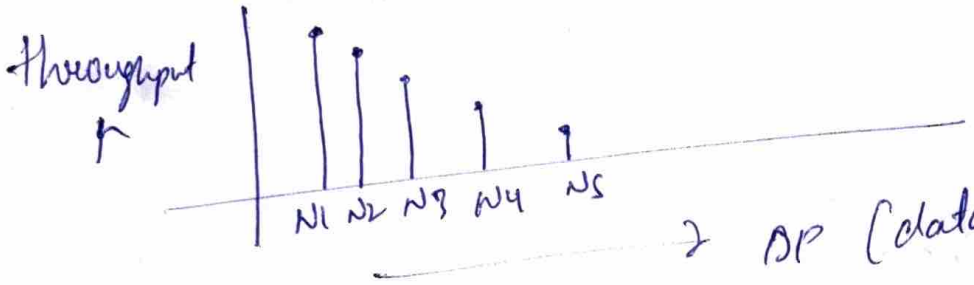
= micro. batch size → batch size seen by each GPU

× grad_acc → gradient accumulation

× dp → data parallelism

takes least number of communication, cause it only communicates gradients

As we keep using multiple nodes, the throughput scaling with data parallelism keeps losing.



Reduce Scatter : + All Gather → All Reduce

All Reduce takes time = 2 × Reduce Scatter

So when ever possible, use only Reduce Scatter

All Gather → after optimizer step

ZeRO 3

Most used data Parallelism than dp

Shards the weights

(FSDP) always needs to ~~share~~ do All Gather

No main changes need to done in dp

but there will be changes in Tensor Parallelism

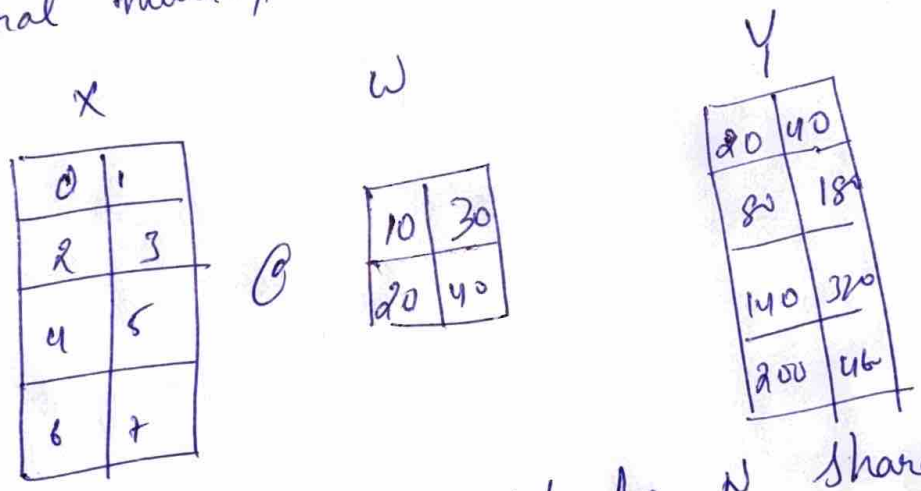
→ like CPU offloading but for GPU

— In single node, GPUs communicate using
NVLink — fastest
— In multiple node, we use EFA or Infini band — little fast
— GPU → CPU communication — slowest

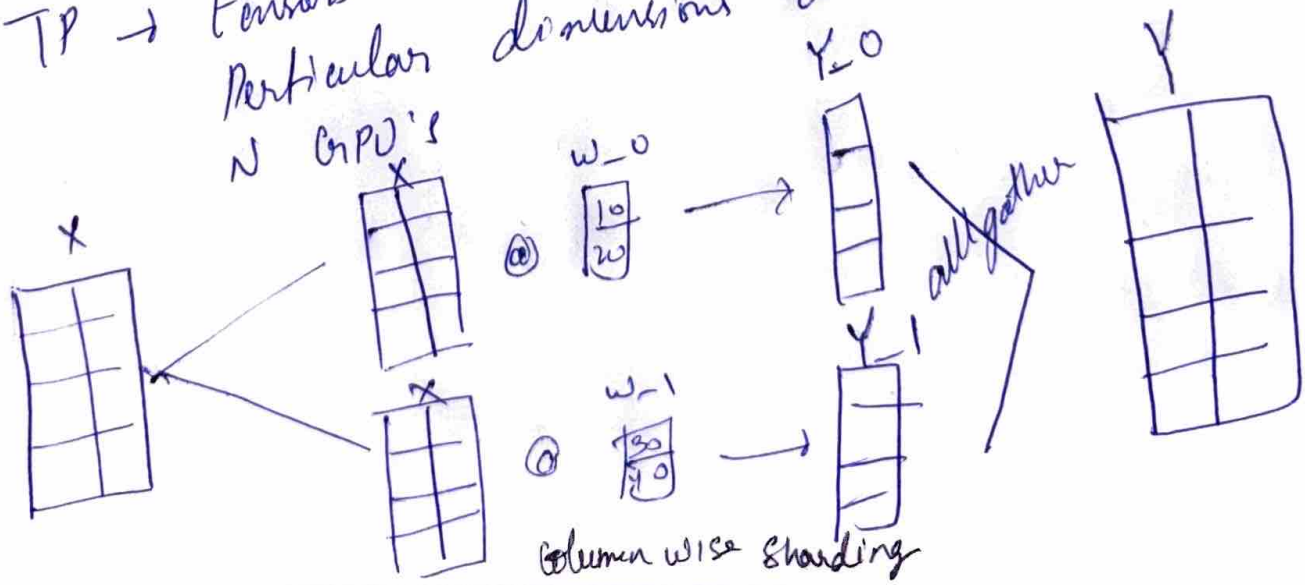
Tensor Parallelism

(data and tensor) parallelism are the toughest.

General multiplication



In TP → tensors are split to N shards, along particular dimensions distributed across N GPU's



Multi-Node GPU inference

→ Tensor parallelism + Pipeline Parallelism

↓ (Number of GPUs to use) (per node)

↓ (Number of nodes)

In Tensor Parallelism → we are trying to shard along the hidden dimension

Shard → dividing the model into layers
→ Parallelizing technique to distribute a large model

DISTRIBUTE

Types
- Tensor Parallelism
- Pipeline Parallelism → If model too large for Single GPU
- Data Parallelism → Model fits but dataset is large
- Optimizer Sharding [ZeRO] → If optimizer states are too big

Stage 1 - only optimizer states are sharded
Stage 2 - optimizer state + Gradients
Stage 3 - Model parameter + optimizer + Gradients

---

f → No operation
f* → All Reduce
g → All Gather
g* → Reduce Scatter Along Sequence dimension

In TP :- Column linear → Shard activations
Row linear → Shard Restore hidden dimension

in TP :- there is only one All Reduce that comes after two linear operation
Also TP is mostly used within a node

Breaking up —
All Reduce = (2 * All Gather) + (2 * Reduce-Scatter)

Context Parallelism

Issue → when we want to scale our sequence length then activations start becoming a very big issue as we have to do activation Recomputation

All to All

Pipe line Parallelism

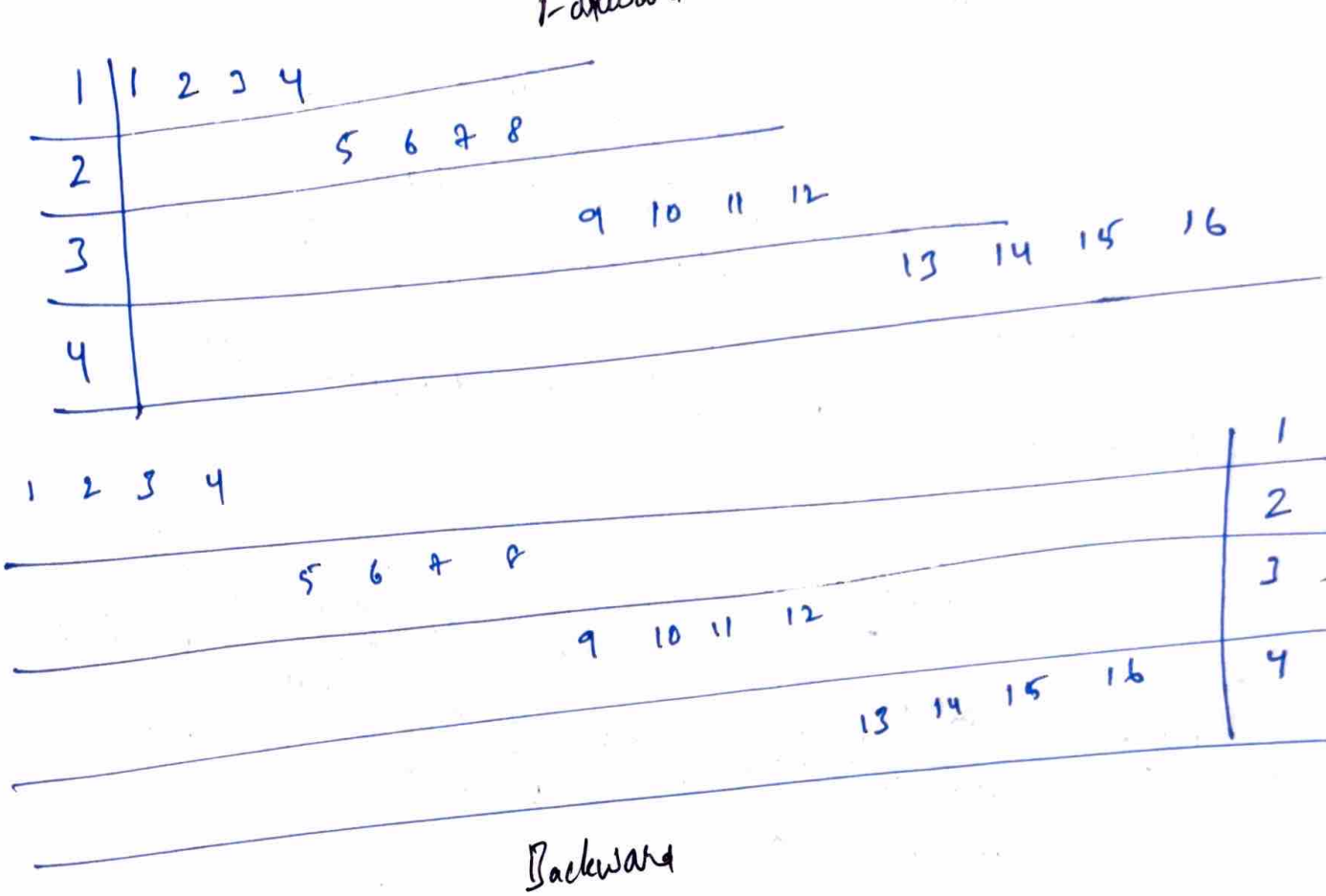
Issue :- parallelising the data/model across Multiple Nodes. as we move towards multiple nodes, bandwidth decrease very quickly.

- least number of collectives, Shards the total model
- Similar to ZeRO3.
- ~~All Gather~~ wh Shard along the layers

GPU1 → 1st 4 layers

GPU2 → Next 4 layers

Ex:- 16 layers across 4 GPUs

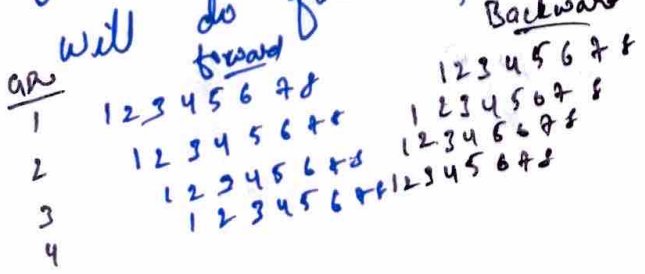


4 layers in each GPU

All Forward & Backward :-

→ first do all forward passes, then only all backward passes.

→ Send a lot of micro batches, first process forward pass for those micro batches and send it to 2nd GPU instead of waiting to get data to GPU 1 again, it will do forward pass for another micro batch.



One Forward One Backward

→ middle / Steady state

do forward for a micro batch → Send activation to next GPU, do forward for another single micro batch → Send activation to next GPU ... so on. In last GPU, micro batch, the same GPU does forward and backward then sends Gradients

→ F B F B F B - - -

Bubble time :- Idle time for GPUs while calculating

- we need to reduce this bubble time, ZERO3 doesn't have bubble time

ZERO and Bubble & Double

has
- forward
- backward
- backward for input
- backward for weight
- overlapped forward and backward

Expert Parallelism

- Implies for only for Experts. only when we have MOE.
- No simple way of defining EP
- EP in this case → Parallelism experts along diff GPUs
- each GPU gets different expert model. Then the attention will be same.
- Each GPU gets different batch of data.

DP :- Along batch dimension
TP :- Along hidden dimension
Seq and Context P :- along Seq dimension
Pipeline P :- Along model layers
EP :- Along model experts